

Review toàn hệ thống VideoLevel (ZK-Stego)

Một lỗi media native, kênh CAVLC giao thức v2, Groth16 bắt buộc khi xác minh

Tài liệu kỹ thuật tiếng Việt

3 tháng 10 năm 2026

Cập nhật 03/10/2026. Bản review này được viết lại toàn bộ theo mã nguồn hiện tại trên nhánh `main` (kể cả các thay đổi chưa commit trong `working tree`). Bản trước (26/09/2026) review hai tuyến song song: lỗi native và một pipeline Python full-video. Pipeline Python đó cùng các benchmark SEC1–SEC10 đã bị gỡ ngày 03/10/2026 (khoảng 18 000 dòng), nên các listing và nhận xét cũ về nó không còn áp dụng. Tài liệu này mô tả: hệ thống làm gì, kiến trúc, điểm mạnh, các vấn đề đã được xác minh và sửa trong hai đợt review 02/10 và 03/10/2026, và các giới hạn còn lại. Nguồn đối chiếu: `doc/he_thong_hoat_dong.md` (mô tả đầy đủ, mục 17 về giới hạn), `README.md` (Status, Known limits), `future_plan.md`, ghi chú đầu `doc/completion_plan.md` và mã nguồn.

Mục lục

1	Phạm vi và cách đọc bản review	3
2	Hệ thống làm gì	3
3	Kiến trúc: một lỗi media và các thành phần quanh nó	3
4	Lớp zero-knowledge	4
4.1	Circuit và ràng buộc bit	4
4.2	Đóng gói và nén proof	5
4.3	Xác minh fail-closed	5
5	Kênh native CAVLC, giao thức v2	6
5.1	Ứng viên và lý do chọn dấu trailing-one	6
5.2	Tách khóa con bằng HKDF	6
5.3	Lịch, khung và whitening	7
5.4	Một giao thức theo đoạn và trích xuất mù	7
6	Native CLI và các sửa lỗi hardening	8
6.1	Lệnh và ranh giới tiến trình	8
6.2	Bộ đọc Annex-B từ stdin	8
6.3	Giới hạn tài nguyên và đường dẫn	9
7	Dịch vụ HTTP/WebSocket	9
7.1	Xác thực trước khi đọc body	9
7.2	Job verify với Groth16 bắt buộc	10
7.3	Các bảo vệ khác	10
8	Bộ lập lịch realtime	11
9	Demo và kiểm thử	11
9.1	Một demo duy nhất	11
9.2	Bộ kiểm thử	11

10 Điểm mạnh của hệ thống hiện tại	12
11 Các vấn đề đã xác minh và sửa trong đợt review 02–03/10/2026	12
12 Contract giữa các bước: input, output và lỗi	12
13 Giới hạn còn lại	13
14 Kết luận	13

1 Phạm vi và cách bản review

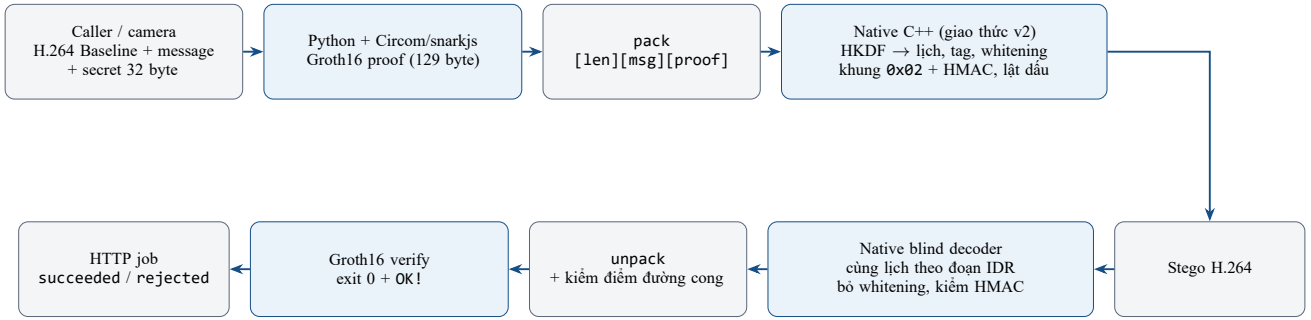
Đây là tài liệu review trạng thái hiện tại của hệ thống VideoLevel, không phải hướng dẫn chạy demo (hướng dẫn nằm ở README.md và demo/README.md). Trọng tâm là các thành phần runtime đang tồn tại: circuit và cầu nối Groth16; lõi native C++ đọc/ghi H.264 CAVLC; kênh xác thực giao thức v2; CLI native; dịch vụ HTTP/WebSocket; bộ lập lịch realtime; demo và bộ kiểm thử.

Các code listing được lấy trực tiếp từ source tree tại thời điểm biên soạn, kèm số dòng gốc để đối chiếu. Mỗi listing được đọc như review code: xác định precondition, theo dữ liệu từng bước, rồi kiểm tra output có đúng hợp đồng ở ranh giới kế tiếp hay không. Thư viện ngoài, mã sinh tự động, test và bảng VLC không được chép lại.

Mọi con số trong tài liệu lấy từ tài liệu đã lưu trong repository (README, doc/he_thong_hoat_dong.md, doc/completion_plan.md, future_plan.md). Bản review này không chạy lại benchmark; các kết quả benchmark đã ghi được nêu kèm điều kiện của chúng (xem mục 13).

2 Hệ thống làm gì

VideoLevel giấu một proof Groth16 (kèm message) trong video H.264 Baseline bằng cách đặt dấu của một số hệ số trailing-one CAVLC trong các frame IDR, sau đó trích xuất mù bằng khóa và bắt buộc xác minh proof. Video là carrier: nó không tự mang khóa, và proof hiện chỉ chứng minh quan hệ message–khóa, không chứng minh tính toàn vẹn của video.



Hình 1: Luồng đầu–cuối hiện tại. Một lõi media native; Python điều phối proof, đóng gói, dịch vụ và demo.

Ở mức khái niệm:

$$\begin{aligned}
 m, k &\longrightarrow \pi, & B &= \text{pack}(m, \text{ser}(\pi)), \\
 (k_s, k_f, k_w) &= \text{HKDF}(k), & F &= [\text{0x02} \parallel |B| \parallel B \parallel \text{HMAC}_{k_f}(\cdot)_{[0:16]}], \\
 V_{\text{stego}} &= \text{embed}_{\text{H.264}}(F \oplus \text{KS}(k_w), \text{schedule}(k_s)), \\
 V_{\text{stego}}, k &\longrightarrow F \longrightarrow (m, \pi) \longrightarrow \text{Verify}(\pi, \text{public}(m, k)).
 \end{aligned}$$

Native channel không tạo proof: caller tạo proof-bearing bytes trước và C++ coi chúng là payload opaque. Khóa không được nhúng trong H.264; bên nhận phải có cùng secret 32 byte và cùng tham số `max_bits_per_IDR`.

3 Kiến trúc: một lõi media và các thành phần quanh nó

Ba lớp có trách nhiệm tách biệt: *media* (bitstream sau khi nhúng vẫn hợp lệ, độ dài mã không đổi), *kênh xác thực* (chỉ người có khóa tìm được vị trí; sai khóa hoặc sửa bit thì HMAC từ chối) và *proof* (quan hệ $c = \text{SHA256}(\text{SHA256}(m) \parallel k)$ được chứng minh bằng Groth16).

Module	Trách nhiệm	Output cho downstream
native/include/zkstego/cavlc_stream.hpp, native/src/cavlc_stream.cpp	Annex-B, SPS/PPS, slice, macroblock, CAVLC; ứng viên dấu trailing-one; HKDF, lịch HMAC, khung v2, whitening; encoder/decoder theo đoạn IDR.	Stego Annex-B hoặc payload đã qua HMAC.
native/tools/blind_bits.cpp, native/tools/cli_io.hpp	CLI <code>zkstego_blind_bits</code> : khóa qua stdin, file hoặc stdin/stdout, đường dẫn Unicode/dài trên Windows.	Exit code (2 = mọi lỗi), video hoặc payload hex.

Module	Trách nhiệm	Output cho downstream
native/tools/inspect.cpp	Công cụ soi duy nhất zkstego_inspect: JSON, --summary, --macroblock, --segments.	Trace NAL/slice/ứng viên, lịch theo đoạn.
native/tests/cavlc_stream_tests.cpp	CTest native.	Bảng chứng traversal/rewrite.
circuits/payload_verify.circom	Ràng buộc bit, SHA-256 commitment, miền độ dài.	Constraint system, 513 public signal.
src/zk_proof.py	Witness, snarkjs prove/verify, nén proof 129 byte, pack/unpack.	Proof bytes; kết quả verify fail-closed.
src/native_blind_contract.py	Bản tham chiếu Python của HKDF/lịch/khung/whitening, segment_schedule.	Test vector chéo ngôn ngữ, lịch cho demo.
src/api/app.py, src/api/native_handlers.py	Lỗi job dùng chung (xác thực, giới hạn, hàng đợi, lưu trữ) và dịch vụ native embed/extract/verify + WebSocket.	Job id, trạng thái, artifact, luồng live.
src/realtime_cavlc.py	Bộ lập lịch realtime có giới hạn theo đoạn.	Báo cáo p95, đoạn lỗi, accepted.
src/h264_tables.py	Bảng VLC CAVLC chuẩn cho phần giải thích của demo.	Giải mã độc lập để đối chiếu.
src/manifest.py, src/key_policy.py, src/trust/demo/	Manifest có chữ ký, chứng chỉ khóa có hạn, giao diện trust thử nghiệm.	Metadata trust nếu caller tích hợp.
	Demo duy nhất terminal_demo.py và các module deep_trace, h264_explain, segment_plan, negative_cases, service_steps, native_io.	Transcript, report.json, kiểm tra [PASS]/[FAIL].
src/runtest/	run_all.py (12 phase + H1), prepare_fixtures.py.	Mã thoát 0/1/2.

Nhận xét review. Hợp nhất về một lõi media là thay đổi kiến trúc lớn nhất của đợt này. Trước 03/10/2026 repository duy trì song song một pipeline Python (src/embedder.py, src/verifier*.py, src/core/, src/bitstream/) với parser, lịch và cách tái dựng khác native, cần file phụ hoặc video gốc khi xác minh. Hai implementation cho cùng một chức năng làm mọi kết luận review phải nhân đôi và dễ lệch nhau. Sau khi gỡ, chỉ còn một parser, một lịch và một giao thức; mã cũ vẫn xem được trong lịch sử git. Phần còn giữ lại từ nhánh cũ là bảng VLC (cho demo), schema manifest và chứng chỉ khóa.

4 Lớp zero-knowledge

4.1 Circuit và ràng buộc bit

Circuit PayloadVerify có 513 public signal (payload_hash[256], commitment[256], payload_length) và private secret[256]. Quan hệ:

$$h = \text{SHA256}(m), \quad c = \text{SHA256}(h||k), \quad 0 < |m| < 1,000,000, \quad x(x-1) = 0 \quad \forall x \in h \cup c \cup k.$$

Mạch hiện có 63 321 constraint.

Listing 1: circuits/payload_verify.circom, dòng 31–89

```

31 template PayloadVerify() {
32   // Public inputs
33   signal input payload_hash[256]; // SHA256 hash of payload (256 bits)
34   signal input commitment[256]; // Expected commitment (256 bits)
35   signal input payload_length; // Payload length in bytes
36
37   // Private inputs
38   signal input secret[256]; // Secret key (256 bits)
39
40   // Booleanity: x * (x - 1) == 0 forces every bit signal into {0, 1}.
41   for (var i = 0; i < 256; i++) {
42     secret[i] * (secret[i] - 1) == 0;
43     payload_hash[i] * (payload_hash[i] - 1) == 0;
44     commitment[i] * (commitment[i] - 1) == 0;
45   }
46

```

```

47 // Intermediate signals
48 signal input_bits[512]; // payload_hash + secret (512 bits)
49
50 // Copy payload_hash to first 256 bits
51 for (var i = 0; i < 256; i++) {
52     input_bits[i] <== payload_hash[i];
53 }
54
55 // Copy secret to last 256 bits
56 for (var i = 0; i < 256; i++) {
57     input_bits[256 + i] <== secret[i];
58 }
59
60 // Compute SHA256(payload_hash || secret)
61 component sha = Sha256(512);
62 for (var i = 0; i < 512; i++) {
63     sha.in[i] <== input_bits[i];
64 }
65
66 // Store computed commitment in intermediate signal
67 signal computed_commitment[256];
68 for (var i = 0; i < 256; i++) {
69     computed_commitment[i] <== sha.out[i];
70 }
71
72 // Verify commitment matches computed value
73 for (var i = 0; i < 256; i++) {
74     commitment[i] === computed_commitment[i];
75 }
76
77 // Constrain the public length to 0 < Length < 1,000,000 bytes.
78 // LessThan(20) range-checks both operands before comparing them.
79 component length_lt_limit = LessThan(20);
80 length_lt_limit.in[0] <== payload_length;
81 length_lt_limit.in[1] <== 1000000;
82 length_lt_limit.out === 1;
83
84 component length_is_zero = IsZero();
85 length_is_zero.in <== payload_length;
86 length_is_zero.out === 0;
87 }
88
89 component main {public [payload_hash, commitment, payload_length]} = PayloadVerify();

```

Nhận xét review. Vòng lặp ở dòng 41–45 là sửa lỗi soundness của đợt 03/10/2026: thành phần Sha256 của circomlib không tự ép đầu vào là 0/1, nên trước đó prover có thể đưa giá trị trường tùy ý vào hàm băm. Sau khi thêm ràng buộc, khóa proving/verification được tạo lại bằng một thiết lập giáo dục cục bộ (circuits/build/DEMO_SETUP_ONLY.txt); proof tạo bằng khóa của mạch cũ không verify với khóa hiện tại. Hash của message tính *ngoài* mạch; mạch không chứa hash video, chỉ số frame hay danh tính camera, nên proof không phải chứng nhận video.

4.2 Đóng gói và nén proof

Payload là `[message_length 4 byte big-endian][message][proof nén 129 byte]`. Proof A, $C \in G_1$, $B \in G_2$ trên BN254 được nén thành tọa độ x (32 byte mỗi phần tử, B gồm hai phần tử $\mathbb{F}_{p^2}$) cộng một byte cờ chứa ba bit chẵn/lẻ của y . Khi giải nén, `bytes_to_proof` từ chối kích thước sai, bit cờ lạ, tọa độ không chính xác ($\geq p$) và điểm không nằm trên đường cong (hoặc twist) bằng `ValueError`. `unpack` kiểm độ dài blob đúng bằng $4 + |m| + |\pi|$. Với khung native tối đa 4096 byte payload, message tối đa $4096 - 4 - 129 = 3963$ byte.

4.3 Xác minh fail-closed

Listing 2: `src/zk proof.py`, dòng 403–416

[illegible]

Nhận xét review. Kết quả verify chỉ là True khi snarkjs vừa thoát mã 0 vừa in dòng kết thúc bằng OK!. Crash, thiếu file hay output lạ đều thành False. Cùng với kiểm điểm khi giải nén, đây là đường fail-closed hai lớp: dữ liệu hỏng bị chặn trước khi tới snarkjs, và snarkjs bất thường không bao giờ được hiểu là hợp lệ. Lưu ý: `verify_proof_for_payload(proof, message, secret)` vẫn nhận secret để tự tính lại commitment, nên trong cấu hình hiện tại verifier phải giữ secret (mục 13).

5 Kênh native CAVLC, giao thức v2

5.1 Ứng viên và lý do chọn dấu trailing-one

Lật `trailing_ones_sign_flag` không đổi độ dài mã, không đổi `TotalCoeff/TrailingOnes` nên không đổi ngữ cảnh `nC`, bảng VLC và cú pháp phía sau; bitstream vẫn hợp lệ. Native lấy một ứng viên mỗi khối residual (dấu trailing-one đầu tiên), thuộc mọi loại khối LumaDC, Luma4x4, ChromaDC, ChromaAC trong slice IDR. Định danh ứng viên có dạng `nal_index:mb:category:block:rbps_bit_offset`, tính theo đầu vào của đoạn (SPS+PPS+IDR), không theo vị trí trong file.

5.2 Tách khóa con bằng HKDF

Listing 3: `native/src/cavlc_stream.cpp`, dòng 1650–1713

```
1650 // ---- Blind channel protocol v2: HKDF key separation, frame tag, whitening.
1651
1652 constexpr std::size_t kSubkeyBytes = 32U;
1653 constexpr std::size_t kFrameHeaderBytes = 3U;
1654 constexpr std::size_t kFrameTagBytes = 16U;
1655 constexpr std::size_t kKeystreamBlockBits = 256U;
1656 constexpr std::string_view kHkdfSalt{"zkstego-cavlc-v2-salt"};
1657 constexpr std::string_view kScheduleInfo{"zkstego/cavlc/v2/schedule"};
1658 constexpr std::string_view kFrameTagInfo{"zkstego/cavlc/v2/frame-tag"};
1659 constexpr std::string_view kWhiteningInfo{"zkstego/cavlc/v2/whitening"};
1660
1661 std::string_view byte_view(const std::vector<std::uint8_t>& bytes) noexcept {
1662     return {reinterpret_cast<const char*>(bytes.data()), bytes.size()};
1663 }
1664
1665 // RFC 5869 HKDF-Extract. The returned PRK is secret: callers own and wipe it.
1666 std::vector<std::uint8_t> hkdf_sha256_extract(
1667     const std::vector<std::uint8_t>& salt,
1668     const std::vector<std::uint8_t>& input_key_material) {
1669     const std::vector<std::uint8_t> zero_salt(32U, 0U);
1670     HmacSha256 hmac(salt.empty() ? zero_salt : salt);
1671     auto digest = hmac.digest(byte_view(input_key_material));
1672     std::vector<std::uint8_t> prk(digest.begin(), digest.end());
1673     secure_wipe(digest.data(), digest.size());
1674     return prk;
1675 }
1676
1677 // RFC 5869 HKDF-Expand. Every intermediate block is wiped; the returned OKM
1678 // is owned (and wiped) by the caller.
1679 std::vector<std::uint8_t> hkdf_sha256_expand(
1680     const std::vector<std::uint8_t>& prk,
1681     const std::string_view info,
1682     const std::size_t length) {
1683     if (prk.size() < 32U) throw std::invalid_argument("HKDF pseudorandom key must be at least 32 bytes");
1684     if (length > 255U * 32U) throw std::invalid_argument("HKDF output length exceeds 255 SHA-256 blocks");
1685     HmacSha256 hmac(prk);
1686     std::vector<std::uint8_t> output;
1687     output.reserve(length + 32U);
1688     SensitiveBytes block_input(std::vector<std::uint8_t>{});
1689     block_input.value().reserve(32U + info.size() + 1U);
1690     std::array<std::uint8_t, 32> block{};
1691     for (std::size_t counter = 1U; output.size() < length; ++counter) {
1692         block_input.value().clear();
1693         if (counter > 1U) block_input.value().insert(block_input.value().end(), block.begin(), block.end());
1694         block_input.value().insert(block_input.value().end(), info.begin(), info.end());
1695         block_input.value().push_back(static_cast<std::uint8_t>(counter));
1696         block = hmac.digest(byte_view(block_input.value()));
1697         output.insert(output.end(), block.begin(), block.end());
1698     }
1699     secure_wipe(block.data(), block.size());
1700     secure_wipe(output.data() + length, output.size() - length);
1701     output.resize(length);
1702     return output;
1703 }
1704
1705 // One v2 subkey of a 32-byte channel secret. The caller wipes the result.
1706 std::vector<std::uint8_t> derive_cavlc_subkey(
1707     const std::vector<std::uint8_t>& secret_key,
1708     const std::string_view info) {
```

```

1709 if (secret_key.size() != 32U) throw std::invalid_argument("native CAVLC key must be 32 bytes");
1710 SensitiveBytes prk(hkdf_sha256_extract(
1711     std::vector<std::uint8_t>(kHkdfSalt.begin(), kHkdfSalt.end()), secret_key));
1712 return hkdf_sha256_expand(prk.value(), info, kSubkeyBytes);
1713 }

```

Nhận xét review. Giao thức v2 (03/10/2026) sinh ba khóa con độc lập từ secret 32 byte bằng HKDF-SHA256 (RFC 5869), salt zkstego-cavlc-v2-salt, info zkstego/cavlc/v2/{schedule, frame-tag, whitening}. Lộ một khóa con không làm lộ secret hay các khóa con khác. Implementation xóa PRK và các khối trung gian (SensitiveBytes, secure_wipe). Python (src/native_blind_contract.py) cài cùng ABI và phase 2 kiểm test vector chéo ngôn ngữ, nên một thay đổi một phía sẽ bị bắt ngay.

5.3 Lịch, khung và whitening

Lịch: $\text{score} = \text{HMAC-SHA256}(k_s, \text{định danh ASCII})$; sắp ứng viên tăng dần theo (score, định danh), lấy $N = \min(\max_bits_per_IDR, \text{số ứng viên của đoạn})$. Khung: $[\text{0x02}][\text{len 2 byte}][\text{payload}][\text{tag 16 byte}]$ với $\text{tag} = \text{HMAC-SHA256}(k_f, \text{version} \parallel \text{len} \parallel \text{payload})_{[0:16]}$, tức 19 byte overhead. Bit nhúng thứ i = bit khung thứ i XOR bit keystream thứ i , với $\text{keystream} = \text{HMAC}(k_w, \text{uint64_be}(0)) \parallel \text{HMAC}(k_w, \text{uint64_be}(1)) \parallel \dots$

Listing 4: native/src/cavlc_stream.cpp, dòng 1746–1785

```

1746 // Keystream block j = HMAC-SHA256(whitening_key, uint64_be(j)).
1747 std::array<std::uint8_t, 32> whitening_block(HmacSha256& hmac, const std::uint64_t block_index) {
1748     std::array<char, 8> counter{};
1749     for (std::size_t index = 0; index < counter.size(); ++index) {
1750         counter[index] = static_cast<char>((block_index >> (56U - 8U * index)) & 0xffU);
1751     }
1752     return hmac.digest(std::string_view(counter.data(), counter.size()));
1753 }
1754
1755 // Returns bits[i] XOR keystream bit (first_bit_index + i). The bit index is
1756 // global to one frame, so a stream session whitens each segment by passing
1757 // the number of frame bits already carried by earlier segments. XOR is an
1758 // involution: the same call un-whitens extracted bits.
1759 std::vector<std::uint8_t> whiten_cavlc_frame_bits(
1760     const std::vector<std::uint8_t>& bits,
1761     const std::vector<std::uint8_t>& whitening_key,
1762     const std::size_t first_bit_index) {
1763     std::vector<std::uint8_t> output;
1764     output.reserve(bits.size());
1765     if (bits.empty()) return output;
1766     HmacSha256 hmac(whitening_key);
1767     std::array<std::uint8_t, 32> block{};
1768     std::uint64_t loaded_block = 0;
1769     bool block_loaded = false;
1770     for (std::size_t index = 0; index < bits.size(); ++index) {
1771         if (bits[index] > 1U) throw std::invalid_argument("authenticated frame bit is invalid");
1772         const auto global_bit = static_cast<std::uint64_t>(first_bit_index) + index;
1773         const auto block_index = global_bit / kKeystreamBlockBits;
1774         if (!block_loaded || block_index != loaded_block) {
1775             block = whitening_block(hmac, block_index);
1776             loaded_block = block_index;
1777             block_loaded = true;
1778         }
1779         const auto bit_in_block = static_cast<std::size_t>(global_bit % kKeystreamBlockBits);
1780         const auto key_bit = static_cast<std::uint8_t>((block[bit_in_block / 8U] >> (7U - bit_in_block % 8U)) & 1U);
1781         output.push_back(static_cast<std::uint8_t>(bits[index] ^ key_bit));
1782     }
1783     secure_wipe(block.data(), block.size());
1784     return output;
1785 }

```

Nhận xét review. Không có whitening thì byte version cố định, trường độ dài nhỏ và cấu trúc proof làm thống kê dấu tại các vị trí được chọn bị lệch. Chỉ số bit keystream là toàn cục trong một khung (first_bit_index), nên khung trải qua nhiều IDR vẫn dùng một dòng khóa liên tục; XOR là phép tự nghịch đảo nên cùng hàm dùng cho bỏ whitening. Giao thức v2 không đọc được file stego tạo bởi v1; đây là thay đổi có chủ đích, không phải lỗi tương thích.

5.4 Một giao thức theo đoạn và trích xuất mù

Chỉ còn một giao thức, theo đoạn (SPS+PPS+1 IDR), dùng chung cho file, job HTTP và luồng live. Mỗi đoạn mang tối đa $\max_bits_per_IDR$ bit; khung trải qua nhiều IDR liên tiếp và chỉ số bit khung đếm liên tục qua các đoạn. Bên nhận

không cần video gốc hay file phụ: parse lại IDR, sinh lại khóa con, tái tạo lịch, bỏ whitening 24 bit đầu để kiểm version = 2 và độ dài $\leq$ giới hạn, rồi đọc đủ khung và so tag hằng thời gian. Sai khóa thường bị từ chối ngay ở byte version.

Nhận xét review. Thông báo lỗi native được phân loại rõ: tag sai, version sai, độ dài vượt giới hạn, lịch thiếu dung lượng, hết video trước khi đọc đủ khung; mọi trường hợp đều thoát mã 2. HMAC hợp lệ chỉ chứng minh payload đến từ người có khóa và không bị sửa; nó *không* thay cho Groth16. zkstego_inspect --segments và segment_schedule cho phép caller tái tạo đúng lịch của encoder, đây là cơ sở để demo kiểm tra từng bit.

6 Native CLI và các sửa lỗi hardening

6.1 Lệnh và ranh giới tiến trình

zkstego_blind_bits có ba nhóm lệnh:

- embed-stream-auth-stdin, extract-stream-auth: đọc/ghi file;
- measure-live-capacity-stdin: in JSON dung lượng ứng viên;
- embed-live-auth-stdin, extract-live-auth-stdin: video vào/ra qua stdin/stdout, dùng cho WebSocket.

Khóa luôn là một dòng 64 ký tự hex trên stdin, không bao giờ qua argv. Các công cụ soi trước đây được gộp vào một công cụ duy nhất zkstego_inspect.

6.2 Bộ đọc Annex-B từ stdin

Listing 5: native/src/cavlc_stream.cpp, dòng 1237–1282

```

1237 void AnnexBnalStreamReader::compact() {
1238     if (head_ == 0U) return;
1239     buffer_.erase(buffer_.begin(), buffer_.begin() + static_cast<std::ptrdiff_t>(head_));
1240     scan_ = scan_ > head_ ? scan_ - head_ : 0U;
1241     head_ = 0U;
1242 }
1243
1244 void AnnexBnalStreamReader::read_more() {
1245     if (eof_) return;
1246     // Drop already-returned bytes at most once per returned NAL.
1247     compact();
1248     auto* source = input_.rdbuf();
1249     if (source == nullptr) throw std::invalid_argument("Annex-B input stream is unavailable");
1250     constexpr std::streamsize chunk_size = 64 * 1024;
1251     // Only request what the stream buffer promises it can deliver without
1252     // blocking (in_avail), so a live pipe never waits to fill a whole chunk.
1253     const auto append_available = [&](const std::streamsize limit) {
1254         const auto available = source->in_avail();
1255         if (available <= 0 || limit <= 0) return;
1256         const auto requested = std::min(available, limit);
1257         const auto old_size = buffer_.size();
1258         buffer_.resize(old_size + static_cast<std::size_t>(requested));
1259         const auto received = source->sgetn(
1260             reinterpret_cast<char*>(buffer_.data() + old_size), requested);
1261         // in_avail() promised `requested` bytes; anything else is a stream
1262         // fault, and accepting zero here could spin forever.
1263         if (received <= 0 || received > requested) {
1264             buffer_.resize(old_size);
1265             throw std::invalid_argument("failed while reading Annex-B input stream");
1266         }
1267         buffer_.resize(old_size + static_cast<std::size_t>(received));
1268     };
1269     if (source->in_avail() > 0) {
1270         append_available(chunk_size);
1271         return;
1272     }
1273     // Nothing buffered: block for a single byte, then drain anything that
1274     // arrived with it.
1275     const auto next = source->sbump();
1276     if (std::char_traits<char>::eq_int_type(next, std::char_traits<char>::eof())) {
1277         eof_ = true;
1278         return;
1279     }
1280     buffer_.push_back(static_cast<std::uint8_t>(std::char_traits<char>::to_char_type(next)));
1281     append_available(chunk_size - 1);
1282 }
```

Nhận xét review. Đợt 02/10/2026 sửa một bộ đọc stdin có chi phí bậc hai: byte đã trả về bị xóa khỏi đầu buffer và NAL đang chờ bị quét lại từ đầu sau mỗi lần đọc. Hiện `compact()` chỉ dồn buffer tối đa một lần cho mỗi NAL trả về, việc quét start code tiếp tục từ `scan_` thay vì quét lại, và `read_more()` chỉ lấy lượng `in_avail()` hứa có sẵn nên pipe live không bị chặn chờ đầy chunk. Cùng với tăng tốc bộ đọc bit, một lần live embed 1080p all-intra giảm từ 30,1 s xuống 2,5 s trên máy phát triển. Con số này là đo trên host, không phải nghiệm thu realtime.

6.3 Giới hạn tài nguyên và đường dẫn

- **Kích thước ảnh từ SPS:** tối đa 36 864 macroblock (MaxFS của level 5.1/5.2, bao phủ 3840×2160) và tối đa 543 MB mỗi chiều; SPS vượt giới hạn bị từ chối thay vì để một SPS độc hại phóng to bộ nhớ decoder.
- **Kích thước đầu vào:** file Annex-B đọc toàn bộ bị giới hạn 1 GiB; `zkstego_inspect` giới hạn 64 MiB; NAL trong luồng bị giới hạn theo cấu hình.
- **Đường dẫn Unicode và dài trên Windows:** hai công cụ vào qua `wmain`, chuyển tham số sang UTF-8 rồi mở file qua `std::filesystem::path/_wopen_s` với tiền tố `\\?\`, nên repository và video có tên tiếng Việt hoặc đường dẫn dài vẫn chạy.

7 Dịch vụ HTTP/WebSocket

7.1 Xác thực trước khi đọc body

Listing 6: `src/api/app.py`, dòng 188–231

```

188     headers.setdefault(key.lower(), value)
189     # Compare against the routed path: with --root-path, uvicorn prefixes scope["path"],
190     # and a raw prefix check would let protected routes skip auth and size limits.
191     path = get_route_path(scope) if scope.get("path") else ""
192     if self.api_token is not None and path.startswith(self._PROTECTED_PREFIX):
193         client = client_host(scope)
194         retry_after = self.auth_limiter.retry_after(client) if self.auth_limiter is not None else 0
195         if retry_after:
196             await self._reject(
197                 send, 429, b'{"detail": "too many failed authentication attempts; retry later"}',
198                 extra_headers=[(b"retry-after", str(retry_after).encode("ascii"))],
199             )
200             return
201         if not bearer_token_matches(headers.get(b"authorization"), self.api_token):
202             failures = self.auth_limiter.record_failure(client) if self.auth_limiter is not None else None
203             _LOGGER.warning(
204                 "API authentication failed: client=%s path=%r failures_in_window=%s", client, path, failures,
205             )
206             await self._reject(
207                 send, 401, b'{"detail": "Bearer token required"}',
208                 extra_headers=[(b"www-authenticate", b"Bearer")],
209             )
210             return
211     if scope.get("method") != "POST" or path not in self._UPLOAD_PATHS:
212         await self.app(scope, receive, send)
213         return
214     if self.max_concurrent_uploads is not None and self._active_uploads >= self.max_concurrent_uploads:
215         await self._reject(
216             send, 503, b'{"detail": "upload capacity reached; retry later"}',
217             extra_headers=[(b"retry-after", b"1")],
218         )
219         return
220     self._active_uploads += 1
221     try:
222         await self._guard_upload(scope, receive, send, headers)
223     finally:
224         self._active_uploads -= 1
225
226     async def _guard_upload(self, scope, receive, send, headers: dict[bytes, bytes]) -> None:
227         maximum_bytes = self.maximum_bytes[scope["path"]]
228         content_length = headers.get(b"content-length")
229         if content_length is not None:
230             try:
231                 if int(content_length) > maximum_bytes:

```

Nhận xét review. FastAPI phân tích (và spool) multipart trước khi dependency của route chạy, nên kiểm token ở dependency nghĩa là server đã nhận body của client chưa xác thực. Middleware RequestBodyGuard kiểm Bearer token trước khi đọc bất kỳ byte body nào. Dòng 196–198 sửa một lỗi bypass: với --root-path, Uvicorn thêm tiền tố vào scope["path"], nên so tiền tố thô có thể để route được bảo vệ bỏ qua xác thực và giới hạn kích thước; hiện guard so trên routed path. Client sai token 10 lần trong 60 s nhận 429 kèm Retry-After (cả HTTP lẫn WebSocket). Token phải dài tối thiểu 32 ký tự, kiểm cả khi khởi động lần trong create_app.

7.2 Job verify với Groth16 bắt buộc

Listing 7: src/api/native_handlers.py, dòng 50–81

```

50 def make_proof_verify_handler(
51     extract_payload: PayloadExtractor, verify_proof: ProofVerifier,
52 ) -> Callable[..., dict[str, Any]]:
53     """Blind-extract ``pack(message, proof)`` and accept it only if its Groth16 proof verifies.
54
55     Returns ``{"valid": True, "message_b64", "payload_bytes"}`` for a verified proof and
56     ``{"valid": False, "reason"}`` when no authenticated payload, a malformed blob or an
57     invalid proof is found. Infrastructure errors (timeouts, missing Node.js/circuits)
58     propagate, so the job service marks the job "failed" rather than "rejected".
59     """
60
61     def verify_handler(*, stego_video_path: str, secret_key: bytes, maximum_payload_bytes: int) -> dict[str, Any]:
62         try:
63             blob = extract_payload(stego_video_path, secret_key, maximum_payload_bytes)
64         except NativePayloadNotAuthenticated:
65             return {"valid": False, "reason": "payload_not_authenticated"}
66         try:
67             message, proof_bytes = unpack(blob)
68             if not message or len(blob) != 4 + len(message) + PROOF_SIZE_BYTES:
69                 raise ValueError("proof payload length mismatch")
70             proof = bytes_to_proof(proof_bytes)
71         except ValueError:
72             return {"valid": False, "reason": "malformed_proof_payload"}
73         if verify_proof(proof, message, secret_key) is not True:
74             return {"valid": False, "reason": "proof_invalid"}
75         return {
76             "valid": True,
77             "message_b64": base64.b64encode(message).decode("ascii"),
78             "payload_bytes": len(blob),
79         }
80
81     return verify_handler

```

Nhận xét review. Trước đợt sửa, route verify của dịch vụ native trả 501 vì không có verifier. Hiện job verify trích mù, unpack, kiểm độ dài chính xác, giải nén điểm, rồi bắt buộc Groth16. Kết quả hợp lệ duy nhất là succeeded kèm message; mọi trường hợp khác là trạng thái rejected với lý do payload_not_authenticated, malformed_proof_payload hoặc proof_invalid. Lỗi hạ tầng (timeout, thiếu Node.js/circuits) được để lan ra nên job thành failed, không bị nhầm thành “proof sai”. verify_proof(...) is not True là so sánh chặt, không chấp nhận giá trị truthy khác.

7.3 Các bảo vệ khác

Cơ chế	Hiện trạng
Trạng thái job	queued, running, succeeded, rejected, failed; rejected là trạng thái kết thúc và client không được coi là thành công.
Upload	Tối đa 4 upload đồng thời; giới hạn kích thước; hạn thời gian upload ZK_STEGO_API_UPLOAD_TIMEOUT_SECONDS (mặc định 120 s).
Hàng đợi	Có giới hạn (mặc định 1 worker + 2 chờ); đầy thì 503 kèm Retry-After.
Lưu trữ	Artifact tải một lần, hết hạn sau 10 phút; job đã xong bị xóa sau ZK_STEGO_API_JOB_RETENTION_SECONDS (mặc định 24 h).
WebSocket	Từ chối trước khi accept nếu thiếu token; đóng sau ZK_STEGO_STREAM_IDLE_TIMEOUT_SECONDS (15 s) không có dữ liệu.
Tiến trình native	Không qua shell, khóa qua stdin, có timeout, stderr được đọc liên tục.

Cơ chế	Hiện trạng
OpenAPI	/docs chỉ bật khi ZK_STEGO_API_DOCS=1.

8 Bộ lập lịch realtime

src/realtime_cavlc.py (RealtimeCAVLCScheduler) xử lý từng đoạn IDR với hàng đợi giới hạn theo số đoạn và số byte (bỏ đoạn cũ nhất khi đầy), epoch tăng đơn điệu và cache proof tối đa 4 epoch. Báo cáo gồm số đoạn bị bỏ, số đoạn lỗi, p95 thời gian patch, p95 độ trễ đầu-cuối và p95 thời gian tạo proof; accepted chỉ đúng khi không có đoạn lỗi và p95 nằm trong ngân sách một frame.

Nhận xét review. Công nghiệm thu được thiết kế fail-closed, nhưng kết quả đo camera thật hiện chưa vượt công 30 FPS (mục 13). Bộ lập lịch là bằng chứng chức năng về giới hạn bộ nhớ và epoch, không phải bằng chứng hiệu năng.

9 Demo và kiểm thử

9.1 Một demo duy nhất

Bộ script demo theo từng bước trước đây đã được gộp vào demo/terminal_demo.py ngày 03/10/2026. Demo dùng cùng giao thức theo đoạn với dịch vụ, và phần deep trace đối chiếu mọi giá trị tự tính với FFmpeg hoặc parser native: pipeline FFmpeg/libx264 từ log verbose; cắt NAL, EBSP→RBSP; toàn bộ SPS/PPS/slice header qua trace_headers; bản đồ loại MB và QP từ decoder FFmpeg; vòng đời đầy đủ của một khối 4×4 (bit header → phần tử CAVLC → hệ số → giải lượng tử → biến đổi ngược → dự đoán intra → pixel, khớp từng mẫu với ảnh FFmpeg -skip_loop_filter all); vị trí bit nhúng trong file và trường payload mà bit đó mang. Sau đó demo chạy các ca từ chối (đổi message, proof sai, thiếu proof, sai khóa, lật một bit mang dữ liệu), job HTTP thật (embed trùng từng byte với CLI, extract, verify → succeeded; verify sai khóa → rejected) và luồng WebSocket. Mọi kiểm tra in [PASS]/[FAIL]; một [FAIL] dừng demo với mã thoát 1.

9.2 Bộ kiểm thử

Ngày 03/10/2026, src/runtest/run_all.py qua 126/126 trên 12 phase (sau khi hợp nhất một lỗi) trên host Windows với fixture từ prepare_fixtures.py; CTest native qua 1/1. Phase camera thật H1 (--hardware) không nằm trong lần chạy này. Mã thoát run_all: 0 = tất cả qua, 1 = có lỗi, 2 = có ca bắt buộc bị bỏ qua.

Phase	File	Nội dung
1	test_zk_proof.py	Proof Groth16: định dạng, nén, verify, fail-closed.
2	test_native_blind_contract.py	C++ và Python khớp HKDF, lịch, khung, whitening, lịch theo đoạn.
3	test_native_cli_fixture.py	CLI nhúng/trích trên fixture 300 frame, decode nghiêm ngặt, sai khóa.
4	test_h264_tables.py	Bảng VLC CAVLC (prefix-free, Kraft, giá trị chuẩn).
5	test_service_api.py	Lỗi job, xác thực trước body, giới hạn thử sai, lưu trữ, job verify, hạn khóa.
6	test_native_http_channel.py	Job HTTP và luồng WebSocket qua Uvicorn thật.
7	test_manifest_security.py	Ký manifest, ràng buộc positions/stego.
8	test_realtime_scheduler.py	Bộ lập lịch realtime.
9	test_benchmark_recorder.py	Kiểm tra fail-closed của bộ ghi benchmark camera.
10	test_trust_interfaces.py	Giao diện trust thử nghiệm (provenance, C2PA, attestation).
11	test_demo_h264_explain.py	Toán giải thích của demo khớp native và pixel FFmpeg.
12	test_demo_smoke.py	Toàn bộ demo chạy không có [FAIL].
H1	test_native_camera_http.py	Camera thật (--hardware).

10 Điểm mạnh của hệ thống hiện tại

- **Một lõi media, một giao thức.** Một parser native, một lịch theo đoạn dùng chung cho file, HTTP và live; không còn hai implementation phải giữ đồng bộ.
- **Nhúng không phá cú pháp.** Kênh dấu trailing-one giữ nguyên độ dài mã và ngữ cảnh CAVLC; output được kiểm bằng decode FFmpeg nghiêm ngặt.
- **Tách khóa và làm trắng.** HKDF tách vai trò lịch/tag/whitening; whitening loại bỏ cấu trúc cố định của khung khỏi thống kê dấu.
- **Trích xuất mù có xác thực.** Không cần video gốc hay sidecar; sai khóa hoặc sửa bit đều bị HMAC từ chối với thông báo phân loại rõ.
- **Xác minh fail-closed.** Kiểm điểm đường cong khi giải nén; snarkjs phải vừa thoát 0 vừa in OK!; job verify chỉ succeeded khi proof đúng.
- **Ranh giới dịch vụ được làm cứng.** Xác thực trước body, throttling, giới hạn kích thước/thời gian/hàng đợi, lưu trữ có hạn, khóa không qua argv hay log.
- **Có thể kiểm chứng từng bit.** zkstego_inspect, segment_schedule và deep trace của demo cho phép đối chiếu mọi bước với FFmpeg và parser native; test chéo ngôn ngữ chặn lệch ABI.

11 Các vấn đề đã xác minh và sửa trong đợt review 02–03/10/2026

Vấn đề	Rủi ro	Cách sửa hiện tại
Circuit không ép tin hiệu hash/commitment/secret là bit	Soundness: prover đưa giá trị trường tùy ý vào Sha256	Ràng buộc $x(x-1) = 0$ cho cả ba mảng; 63 321 constraint; khóa demo cục bộ mới.
Verify dựa vào mã thoát snarkjs	Crash/output lạ có thể bị hiểu nhầm	Yêu cầu exit 0 và OK!; bytes_to_proof kiểm tọa độ chính tắc và điểm trên đường cong.
Một khóa cho lịch và tag; khung không làm trắng	Lộ một vai trò ảnh hưởng vai trò khác; thống kê dấu lệch do header cố định	Giao thức v2: HKDF ba khóa con, version 0x02, whitening bằng keystream HMAC.
Nhiều chế độ nhúng cho file và live	Lịch khác nhau giữa các đường vào	Một giao thức theo đoạn IDR cho file, HTTP và live.
Hai lõi media (native và Python)	Hai implementation lệch nhau, Python cần sidecar/video gốc	Gỡ pipeline Python và benchmark SEC (khoảng 18 000 dòng); một lõi native.
Nhiều công cụ soi ròi rác	Kết quả soi không nhất quán	Một công cụ zkstego_inspect.
Token kiểm sau khi parse multipart	Client chưa xác thực đẩy được body lên server	RequestBodyGuard xác thực trước khi đọc body.
Kiểm tiền tố thô khi chạy sau --root-path	Bypass xác thực và giới hạn kích thước	So trên routed path.
Token yếu, không giới hạn thử sai	Đoán token	Token ≥ 32 ký tự; 10 lần sai/60 s $\rightarrow$ 429 + Retry-After.
Route verify native trả 501	Không có đường xác minh đầu–cuối qua dịch vụ	Job verify với Groth16 bắt buộc; trạng thái rejected riêng.
Job và upload không có hạn	Tích tụ dữ liệu, upload treo	Retention 24 h, artifact 10 phút, hạn upload qua biến môi trường.
Bộ đọc stdin bậc hai	Live embed chậm theo kích thước NAL	Compaction mỗi NAL, quét tiếp từ scan_; 1080p live embed 30,1 s $\rightarrow$ 2,5 s trên máy phát triển.
Kích thước SPS và file không giới hạn	SPS độc hại gây cấp phát lớn	36 864 MB, 543 MB/chiều; file 1 GiB; inspect 64 MiB.
Đường dẫn Unicode/dài trên Windows	CLI lỗi trong thư mục có tên tiếng Việt	wmain + UTF-8 + tiền tố đường dẫn dài.
Demo tách thành nhiều script	Khó duy trì, lặp mã	Một terminal_demo.py với deep trace đối chiếu FFmpeg.

12 Contract giữa các bước: input, output và lỗi

Ranh giới	Input	Output	Lỗi riêng
ZK bridge	message + secret 32 byte + khóa demo	proof + public signals	Thiếu Node/circuit khác proof sai.
Nén proof	proof dictionary / 129 byte	bytes / proof đã kiểm điểm	ValueError khi điểm hoặc cò sai.
pack/unpack	message + proof	[4 byte len][message][proof]	Độ dài sai, thiếu proof.
Native encoder	Annex-B + secret + payload + max_bits_per_IDR	Stego Annex-B	Cú pháp ngoài profile, thiếu dung lượng (mã 2).
Native decoder	Stego + cùng secret + giới hạn payload + cùng max_bits_per_IDR	Payload đã qua HMAC	Tag/version/độ dài sai, video kết thúc sớm (mã 2).
Groth16 verify	proof + message + secret	True chỉ khi exit 0 và OK!	Proof sai khác lỗi hạ tầng.
Job API	upload + Bearer + trường form	job id, trạng thái, artifact	401, 413, 429, 503; rejected khác failed.

13 Giới hạn còn lại

- **Không ràng buộc proof với video hay phiên.** Khung và proof không chứa định danh video hay bộ đếm, nên một payload có thể bị phát lại sang video khác dùng cùng khóa. Hướng khắc phục (commitment theo đoạn, chuỗi phiên, final seal) nằm trong `future_plan.md`.
- **Verifier phải giữ secret.** Kênh native dùng khóa con HKDF, nhưng mạch vẫn dùng secret gốc và verifier phải biết secret để tính lại commitment; ai có khóa cũng tạo được proof.
- **Realtime chưa được nghiệm thu.** Lần đo camera thật đã sửa cách đếm frame nguồn đạt 7,608 FPS so với công 30 FPS. Con số này đo trước đợt tăng tốc bộ đọc stdin/bit ngày 02/10/2026 và chưa được đo lại; chưa có kết quả trên thiết bị edge.
- **Kết quả benchmark đã ghi đã cũ.** Các kết quả `_new`, camera và media trong repository được tạo với giao thức v1 và mạch chưa có ràng buộc bit; phải chạy lại trước khi trích dẫn cho phiên bản này. Trong run đã ghi 20260924T150707Z_2be60c, 13/27 case có PSNR frame đã sửa dưới mốc 40 dB (thấp nhất 32,76 dB).
- **Trusted setup chỉ dùng cho demo.** Khóa proving/verification là thiết lập giáo dục cục bộ, không phải ceremony cho triển khai.
- **Profile H.264 bị giới hạn.** Chỉ Baseline, CAVLC, progressive, 4:2:0, một slice mỗi frame bắt đầu ở MB 0, không I_PCM, một SPS/PPS id; chỉ IDR mang dữ liệu. Đây không phải hỗ trợ H.264 tổng quát.
- **Không tương thích ngược.** Giao thức v2 không đọc được stego v1; proof tạo bằng khóa mạch cũ không verify với khóa hiện tại.
- **Trust và chính sách khóa chưa được cường chế ở đường native.** `src/key_policy.py`, `src/manifest.py` và `src/trust/` (gồm MockTEESigner dùng HMAC) là giao diện thử nghiệm; CLI native và dịch vụ nhận secret thô, không tự kiểm chứng chỉ hay hạn khóa. Bearer token của API và secret nhúng là hai credential khác nhau.
- **Triển khai một tiến trình.** Hàng đợi job nằm trong tiến trình và có lease thư mục làm việc, nên dịch vụ phải chạy với một worker Uvicorn.

14 Kết luận

Tại ngày 03/10/2026, VideoLevel là một prototype nghiên cứu có một lõi media native C++, một giao thức kênh v2 (HKDF, khung 0x02, whitening) dùng chung cho file, HTTP và live, một circuit đã ràng buộc bit, xác minh fail-closed và một dịch vụ HTTP/WebSocket đã được làm cứng ở ranh giới xác thực và tài nguyên. Bộ kiểm thử 12 phase qua 126/126 và demo đối chiếu từng bước với FFmpeg cung cấp bằng chứng chức năng trên host phát triển.

Không được suy ra rằng: proof chứng minh video nguyên vẹn hay nguồn camera; HMAC hợp lệ thay cho Groth16; hệ thống đạt realtime; kết quả benchmark đã ghi áp dụng cho giao thức v2; mọi profile H.264 được hỗ trợ; hoặc khóa demo dùng được cho triển khai. Ưu tiên tiếp theo, theo `future_plan.md`: ràng buộc proof với nội dung đoạn video và phiên ghi hình, bỏ yêu cầu verifier giữ secret, đo lại realtime và benchmark với giao thức v2, rồi đo trên thiết bị triển khai.